Ma démo MaaS, pas à pas

Du modèle Hugging Face à son premier appel API. Chaque fiche donne les gestes, les champs et le résultat à montrer. Compter 20 à 30 minutes pour une première répétition, puis 15 à 20 minutes une fois le parcours maîtrisé, hors préparation de la plateforme.

> « Notre équipe Data Science veut utiliser un assistant. Je vais déployer le modèle, donner un accès à l'équipe, puis montrer sa consommation. »

Avant le direct - préparer mes deux rôles

Le parcours : créer un projet → déployer le modèle → définir une souscription et une autorisation → créer une clé → appeler le modèle → voir les tokens.

Mes deux fenêtres de navigateur :

- **ADMIN : fenêtre normale.** Je crée les ressources et je consulte les métriques.
- **DATA SCIENTIST : fenêtre privée / InPrivate, compte `demo-user`.** Je vois ma souscription et je crée ma clé. Un nouvel onglet normal partage la session admin : utiliser une vraie fenêtre privée.

Mon terminal : l'application où je colle les commandes, sur mon ordinateur. Sur le poste actuel, ouvrir un terminal et exécuter :

```
cd "/home/lgheziel/Summit Connect maas/repo"
make endpoints
```

`cd` ouvre le dossier du dépôt. `make endpoints` affiche les adresses OpenShift, OpenShift AI et MaaS de la sandbox connectée. Sur un autre poste, utiliser le dossier où le dépôt a été cloné.

À vérifier avant de présenter :

- La plateforme est préparée avec `make prepare`, selon le guide de préparation. Le GPU est libre pour la création du modèle.
- Le compte `demo-user`, son mot de passe et le groupe `demo-team` sont prêts.
- Python, curl et make sont disponibles dans le terminal utilisé pour l'appel.
- Sur une nouvelle sandbox, noter les nouvelles adresses données par `make endpoints`. Les commandes avec une adresse complète dans cette feuille correspondent à la sandbox validée le 5 novembre 2026.

Choisir mon point de départ : si `make demo` a déjà été exécuté, le projet et le modèle existent. Pour apprendre à utiliser la clé, commencer à l'étape 5. Pour montrer toutes les créations, préparer une sandbox avec `make prepare` ; voir le guide pour la remise à zéro d'une démo existante. Le reset du kit conserve le projet : une nouvelle sandbox permet aussi de montrer sa création.

1 - ADMIN : créer mon projet

Où : navigateur normal, connecté en administrateur dans la console OpenShift.

Je fais :

1. En haut, ouvrir le **lanceur d'applications**, puis **Red Hat OpenShift AI**. Si ce lien manque, ouvrir l'adresse OpenShift AI obtenue avec `make endpoints`.
2. Cliquer **Projects** → **Create project**.
3. Remplir **Name** : `maas-demo`. Vérifier que le nom de ressource est également `maas-demo`.
4. Remplir **Description** : `Un assistant partagé pour l'équipe Data Science`.
5. Cliquer **Create**, puis ouvrir le projet.

Un geste dans le terminal admin : autoriser la gateway MaaS à utiliser les routes de ce projet. Depuis le dossier du dépôt, copier cette commande complète puis appuyer sur **Entrée** :

```
oc label namespace maas-demo \
  maas.opendatahub.io/gateway-access=true \
  app.kubernetes.io/part-of=maas-demo --overwrite
```

Je montre : le projet `maas-demo`, encore sans modèle. La commande termine sans erreur ; elle ajoute les labels nécessaires au parcours MaaS.

> « Le projet est l'espace qui regroupe mon déploiement. Je l'autorise à exposer le modèle par notre gateway MaaS, le point d'entrée commun des appels. »

Si le projet existe déjà : l'ouvrir pour une répétition des étapes suivantes. La création d'un projet vide nécessite l'état de départ prévu avant le direct.

2 - ADMIN : déployer le modèle Hugging Face

Où : Projects → maas-demo → Deployments → Deploy model.

> « Hugging Face héberge les fichiers du modèle. OpenShift va les télécharger, puis exécuter l'inférence sur notre GPU. »

Écran Preconfigure deployment : sélectionner `maas-demo` si demandé, puis **Next**.

Écran Model details - la source :

- **Model location** : `URI`.
- **URI** : `hf://Qwen/Qwen2.5-0.5B-Instruct`.
- Décocher **Create a connection to this location** : ce modèle public ne demande pas de secret Hugging Face.
- **Model type** : `Generative AI model (Example, LLM)`.
- Cliquer **Next**.

Écran Model deployment - les ressources :

- **Model deployment name** : `demo-llm`.
- **Description** : `Assistant Qwen partagé avec l'équipe Data Science`.
- **Deployment method** : **LLM inference service with llm-d**.
- **Hardware profile** : **MaaS - 1 GPU**, préparé par le kit.
- Dans **Customize resource requests and limits** : CPU demandé `1`, limite `4` ; mémoire demandée `4 GiB`, limite `8 GiB` ; GPU `1`.
- **Replica count** : `1`, puis **Next**.

Écran Advanced settings - la publication :

- Cocher **Publish as MaaS**. **Require token authentication** devient décoché et désactivé : l'authentification sera assurée par MaaS.
- Laisser **Add as AI asset endpoint** décoché pour ce parcours API.
- Cocher **Add custom runtime environment variables**, puis **Add variable**.
- **Name** : `VLLM_ADDITIONAL_ARGS`.
- **Value** : copier la ligne suivante, puis **Next**.

```
--dtype=half --max-model-len=2048 --gpu-memory-utilization=0.60 --enforce-eager
```

Écran Review : vérifier le projet, l'URI Qwen, une réplique, le GPU et **MaaS endpoint: Yes**, puis cliquer **Deploy model**.

Je montre : le déploiement qui passe à **Ready** après le chargement. La publication MaaS peut encore attendre sa souscription et son autorisation : elles arrivent aux étapes 3 et 4.

3 - ADMIN : créer la souscription

Où : Settings → MaaS governance → Subscriptions → Create subscription.

Une souscription, c'est quoi ? Elle associe des bénéficiaires à un modèle et fixe leur limite de consommation.

Je remplis :

- **Name** : `demo-quota`.
- **Description** : `Équipe Data Science - 2 000 tokens par minute`.
- **Priority** : `10`. Pour cette démo, les clés seront liées explicitement à cette souscription.
- **Groups** : sélectionner **demo-team**.

J'ajoute le modèle et la limite :

1. Dans **Models**, cliquer **Add models**.
2. Chercher `demo-llm` et vérifier le projet **maas-demo**.
3. Cliquer **Add model** sur sa ligne, puis **Add models** pour valider la sélection.
4. Sur la ligne du modèle, cliquer **Add token limit**.
5. Dans **Edit subscription token limits**, saisir **2000** tokens per **1**, puis choisir **minute**. Vérifier l'unité : le défaut peut être **hour**.
6. Cliquer **Save**.

Avant de créer : décocher **Create authorization policy**. Nous allons créer et expliquer cette autorisation séparément à l'étape suivante.

Cliquer **Create subscription**.

Je montre : la souscription `demo-quota`, le groupe `demo-team`, le modèle `demo-llm` et la limite **2,000 / 1 minute**.

> « Cette équipe dispose d'un budget de 2 000 tokens par minute pour ce modèle. Un token correspond à une unité de texte traitée par le modèle ; un appel consomme plusieurs tokens. »

Passage à l'étape suivante : la souscription définit le cadre de consommation. Je dois aussi autoriser l'équipe à atteindre le modèle.

4 - ADMIN : donner l'autorisation

Où : console **OpenShift**, avec le compte admin, bouton **+** → **Import YAML**.

Pourquoi cet écran ? Le YAML est un formulaire écrit : chaque ligne nomme un champ. Cette méthode conserve les noms exacts utilisés par le kit, quelle que soit la disponibilité du formulaire de policy dans OpenShift AI.

Je fais : copier le bloc suivant dans l'éditeur, expliquer les trois champs ci-dessous, puis cliquer **Create**.

```
apiVersion: maas.opendatahub.io/v1alpha1
kind: MaaSAuthPolicy
metadata:
  name: demo-access
  namespace: models-as-a-service
spec:
  subjects:
    groups:
      - name: demo-team
  modelRefs:
    - name: demo-llm
      namespace: maas-demo
```

Les champs à expliquer :

- `metadata.name: demo-access` : le nom de mon autorisation.
- `subjects.groups: demo-team` : l'équipe qui reçoit le droit d'accès.
- `modelRefs` : le modèle `demo-llm` dans le projet `maas-demo`.

Puis, dans le terminal admin, appliquer les droits de découverte nécessaires à l'interface de cette build :

```
oc apply -f manifests/govern/user-rbac.yaml
```

Je montre : dans **OpenShift AI** → **Settings** → **MaaS governance** → **Authorization policies**, retrouver `demo-access`. Vérifier son état **Active** et la publication du modèle **Ready**. Au besoin, vérifier depuis le terminal :

```
oc get maasauthpolicy demo-access -n models-as-a-service
oc get maasmodelref demo-llm -n maas-demo
```

> « La souscription fixe le budget. L'autorisation donne le droit d'appeler ce modèle. Les deux ciblent la même équipe. »

5 - DATA SCIENTIST : me connecter et créer ma clé

Avant le partage d'écran, sur ton poste : cette commande fonctionne depuis n'importe quel dossier, même si le terminal affiche `~` :

```
cat "/home/lgheziel/Projects/rhoai-maas-demo/secrets/demo-user.password"
printf '\n'
```

Copier uniquement le mot de passe affiché dans **Password**. `printf` ajoute un retour à la ligne pour le séparer de `lgheziel@...`. Le chemin court `cat secrets/demo-user.password` exige d'être dans le dossier du dépôt : depuis `~`, il échoue. Le fichier est privé, absent de GitHub. S'il manque, voir le guide, section 4. Ouvrir ensuite un terminal neuf pour filmer les appels, sans le mot de passe à l'écran.

Où : une **nouvelle fenêtre de navigation privée / InPrivate**. Garder la fenêtre normale ouverte pour l'admin.

Je me connecte :

1. Ouvrir l'adresse OpenShift AI affichée par `make endpoints`.
2. Choisir **corp-demo** sur cette sandbox ; sur une autre, utiliser le fournisseur préparé pour `demo-user`.
3. **Username** : `demo-user`. **Password** : la valeur du fichier ci-dessus.
4. Cliquer **Log in** et vérifier le compte affiché en haut à droite : **demo-user**.

Je découvre mon accès et remplis le formulaire :

1. Dans le menu latéral, ouvrir **Gen AI studio → API keys**.
2. Onglet **Subscriptions**, puis cliquer **demo-quota**.
3. Montrer le modèle et **2,000 / 1 minute**, puis cliquer **Create API key**.
4. **Subscription** : vérifier **demo-quota**, préremplie depuis cette fiche.
5. **Name** : `demo-notebook`.
6. **Description** : `Mon premier appel à l'assistant`.
7. **Expiration → On date** : choisir une date couvrant la démo, par exemple J+7, dans les bornes affichées. Le format est `AAAA-MM-JJ`.
8. Cliquer **Create API key**, puis copier la **valeur de la clé** affichée une seule fois. Garder cette fenêtre ouverte jusqu'à son enregistrement à l'étape 6.

Je montre : la souscription accessible et le formulaire rempli. Masquer la valeur secrète lors d'un partage d'écran ou d'un enregistrement.

> « Mon mot de passe sert à me connecter à l'interface. Ma clé API va permettre à mon programme d'appeler le modèle avec les droits de ma souscription. »

Si l'onglet Subscriptions manque : passer par le menu latéral **Gen AI studio → API keys**. Le raccourci d'accueil **Manage API keys** peut ouvrir une ancienne page.

6 - DATA SCIENTIST : enregistrer ma clé, simplement

Où : le terminal sur mon ordinateur. Cette étape n'exige pas de droits administrateur OpenShift.

Enregistrer la clé signifie : la conserver dans un fichier privé pour que le programme puisse la lire à chaque appel. Je le fais une seule fois par clé.

1. Ouvrir le dossier du dépôt. Sur le poste actuel, copier cette commande puis appuyer sur **Entrée** :

```
cd "/home/lgheziel/Summit Connect maas/repo"
```

Sur un autre poste, utiliser le dossier de son clone. Le bon dossier contient le fichier `Makefile` et le répertoire `scripts`.

2. Lancer l'enregistrement :

```
make save-key
```

3. Répondre à la première invite du terminal :

```
Clé de demo-quota (Entrée pour conserver si déjà enregistrée) :
```

Coller la clé copiée à l'étape 5 avec **Ctrl + Maj + V** ou clic droit → **Coller**, puis appuyer sur **Entrée**. Coller uniquement la valeur de la clé, sans ajouter le mot `Bearer`. Pour ce premier essai, coller la nouvelle clé ; appuyer sur Entrée sans valeur conserve seulement une clé déjà enregistrée.

Rien ne s'affiche pendant le collage : c'est normal. La saisie est masquée, même si la clé est bien reçue.

4. Répondre à la deuxième invite :

```
Clé de demo-low-quota (Entrée pour ignorer) :
```

Appuyer simplement sur **Entrée**. Cette seconde clé ne sert qu'au bonus sur le petit quota.

Je vérifie le message final :

```
Clé(s) enregistrée(s) dans secrets/, sans affichage.
```

La clé principale se trouve maintenant dans `secrets/demo-user.apikey`, un fichier local exclu de Git. Pour la remplacer plus tard, relancer `make save-key` avec la nouvelle clé.

> « Ma clé est enregistrée. Le programme va la lire automatiquement ; je peux maintenant tester le modèle. »

7 - DATA SCIENTIST : faire mon premier appel

Où : le même terminal, toujours dans le dossier du dépôt.

Je fais : pour la sandbox actuelle, copier cette commande complète puis appuyer sur **Entrée** :

```
MAAS_URL=https://maas.apps.ocp.mqgbd.sandbox840.opentlc.com make call
```

Ce que cette ligne fait : `MAAS_URL` indique l'adresse du service. `make call` lit ma clé enregistrée, découvre mon modèle et lui pose la question : « En une phrase simple, explique à quoi sert une API. »

Sur une nouvelle sandbox, remplacer l'adresse après `MAAS_URL=` par celle fournie par l'admin avec `make endpoints`. Dans l'adresse, utiliser le service **MaaS**, pas la console OpenShift AI.

J'attends que le terminal affiche la réponse. Le programme utilise la clé de `demo-user` ; il n'a pas besoin d'une connexion admin au cluster.

Je repère trois éléments dans le résultat :

- `"http": 200` : l'appel a réussi.
- `"answer"` : le texte produit par Qwen.
- `"usage"` : les tokens de l'appel. `prompt_tokens` compte l'entrée, `completion_tokens` la sortie, `total_tokens` leur somme.

Exemple réellement mesuré pendant la validation : 43 tokens d'entrée + 64 tokens de sortie = **107 tokens**. Les nombres et le texte peuvent varier ; montrer ceux de l'appel en cours. Le client limite la sortie à 64 tokens, ce qui peut couper la fin d'une réponse.

> « L'appel a réussi avec ma clé personnelle. Je vois la réponse du modèle et combien de tokens cet appel a consommés. »

Pour refaire un appel : relancer exactement la même commande. Il n'est pas nécessaire de recréer ni de réenregistrer la clé.

Ce que je montre au public : le résultat de l'appel dans le terminal. Je garde le fichier de clé fermé.

8 - ADMIN : montrer les tokens réellement consommés

Où : revenir dans la **fenêtre normale admin**, puis **Observe & monitor → Dashboard → MaaS démo**.

Avant de générer du trafic : noter le nombre affiché dans **Tokens consommés**. Noter aussi **Appels autorisés**.

Dans le terminal utilisé par le data scientist, envoyer cinq appels avec la clé principale :

```
MAAS_URL=https://maas.apps.ocp.mqgbd.sandbox840.opentlc.com make traffic
```

Sur une nouvelle sandbox, remplacer l'adresse comme à l'étape 7. Le terminal affiche cinq résultats et leurs tokens.

Revenir au navigateur admin, attendre 30 à 60 secondes pour la collecte des métriques, puis cliquer **Refresh**.

Je montre :

- **Tokens consommés** : le compteur a augmenté.
- **Appels autorisés** : les cinq appels sont comptabilisés.
- **Tokens par souscription sur 5 minutes** : la courbe porte le nom du modèle et de la souscription.

Ma preuve : comparer l'augmentation du compteur à la somme des `total_tokens` des cinq réponses. Lors de la validation, cinq appels ont produit **535 tokens** et le compteur est passé de **963 à 1 498**, soit **+535**. Utiliser les valeurs du direct, sans recopier celles de l'exemple. Pour une comparaison exacte, éviter d'autres appels simultanés à la même souscription.

> « L'administrateur voit la consommation réelle de cette souscription. Il peut suivre l'usage du modèle partagé par l'équipe. »

À expliquer si nécessaire : MaaS démo est une vue ajoutée par ce dépôt, alimentée par les vrais compteurs. Le dashboard natif **Usage** présente une incompatibilité de métriques sur cette build. Les compteurs repartent de zéro au redémarrage de Limitador ; ils ne représentent pas une facture financière.

Phrase de fin : « J'ai déployé un modèle, donné un accès gouverné à une équipe, laissé le data scientist créer sa clé et mesuré sa consommation. »

Bonus et aide - seulement si nécessaire

Bonus : montrer un quota dépassé. Le parcours principal est terminé ; cette séquence est facultative.

1. Côté admin, préparer `demo-low-quota` avec le même modèle et le même groupe, mais **50 tokens par minute**. Depuis le dépôt :

```
oc apply -f manifests/govern/low-quota.yaml
```

2. Côté `demo-user`, créer une seconde clé liée à **demo-low-quota**, avec le nom `demo-quota-test`.
3. Relancer `make save-key` : à la première invite, appuyer sur **Entrée** pour conserver la clé principale déjà enregistrée. À la seconde, coller la nouvelle clé de petit quota, puis **Entrée**.
4. Lancer :

```
MAAS_URL=https://maas.apps.ocp.mqgbd.sandbox840.opentlc.com make quota-test
```

Je montre : un appel accepté, puis **HTTP 429**. Le premier appel peut dépasser les 50 tokens avant que sa réponse soit comptabilisée ; les appels suivants sont alors limités. Après collecte, **Appels limités** augmente dans le dashboard admin.

Si je suis bloqué :

- **`No rule to make target`** : vérifier le dossier du terminal ; revenir à l'étape 6 et refaire `cd`.
- **Fichier `demo-user.apikey` introuvable** : faire `make save-key`, puis relancer l'appel.
- **`Clé MaaS attendue`** : recopier la valeur complète de la clé API, puis refaire `make save-key`.
- **HTTP 401 / 403** : vérifier l'expiration de la clé et sa souscription ; côté admin, vérifier le groupe et `demo-access`.
- **HTTP 429 avec la clé principale** : attendre la prochaine fenêtre d'une minute et réessayer ; vérifier que la clé est liée à `demo-quota`.
- **Pas de métriques** : faire un appel réussi, attendre la collecte, puis Refresh ; contrôle admin de secours : `make usage`.
- **Modèle en Pending** : vérifier que le GPU est libre ; diagnostic détaillé dans le guide.

Repère final : mot de passe → connexion à l'interface ; souscription + autorisation → accès gouverné ; clé API → appel du modèle ; dashboard → consommation réelle.

Les intitulés et comportements ont été vérifiés sur RHOAI 3.5.2. Les essais réalisés et leurs limites figurent dans Validation.

Pour enregistrer ce parcours sans voix, suivre le guide vidéo OBS et Kdenlive.